Ejercicio: "Gestor de Biblioteca"

Contexto

Una universidad quiere digitalizar la gestión de su biblioteca. Necesitan un sistema que permita registrar libros y revistas (ambos son publicaciones), administrar préstamos a socios, y guardar toda la información en una base de datos relacional usando JDBC.

Requisitos técnicos

1. Genericidad
    - Crea una interfaz genérica Repositorio<T, ID> con operaciones *guardar*(T), *buscarPorId*(ID), *listarTodos*(), *eliminar*(ID).
    - Implementa al menos *RepositorioLibro*, *RepositorioSocio* y *RepositorioPrestamo* que implementen dicha interfaz.
    - Crea una clase genérica *Cache*<T, ID> que mantenga en memoria (por ejemplo con un *HashMap*<ID, T>) los objetos consultados recientemente, con un método *obtenerOCargar*(*ID*, *Supplier*<T>).
2. Colecciones
    - Usa List, Map, Set según convenga. Por ejemplo:
        - *Set*<*Publicacion*> para evitar duplicados en el catálogo.
        - *Map*<*Integer*, *List*<*Prestamo*>> para agrupar préstamos por socio.
        - *Queue*<*Prestamo*> para gestionar una lista de espera de préstamos cuando un libro no está disponible.
    - Ordena el catálogo por título y por año usando *Comparator* y *Collections.sort / List.sort.*
3. Excepciones
    - Define una jerarquía propia: *BibliotecaException* como base, y subclases *PublicacionNoEncontradaException*, SocioNoEncontradoException, *LibroNoDisponibleException*, *DatosInvalidosException*.
    - Valida datos (por ejemplo, ISBN con longitud correcta, año > 0, socio con email válido) y lanza *DatosInvalidosException* cuando corresponda.
    - Captura *SQLException* y envuélvela en una excepción de dominio (*PersistenciaException*) para no filtrar detalles técnicos a las capas superiores.
    - En los métodos que abren conexiones, *PreparedStatement* o *ResultSet*, ciérralos manualmente en un bloque *finally* (con sus propios

try/catch internos para no enmascarar la excepción original). Esto refuerza la importancia de liberar recursos aun cuando ocurra un error.

4. Persistencia con JDBC
    - Crea las tablas *publicaciones*, *libros*, *revistas*, *socios* y *prestamos* (puedes usar herencia por tabla o una sola tabla con discriminador; se sugiere una tabla publicaciones con columna tipo).
    - Implementa un *ConnectionManager* que lea los datos de conexión desde un archivo db.properties (URL, usuario, contraseña).
    - Realiza operaciones CRUD con *PreparedStatement* (evita *Statement* para prevenir inyección SQL).

Modelo de dominio sugerido

```
Publicacion (abstracta)  -> id, titulo, anio, disponible
  Libro            -> isbn, autor, numPaginas
  Revista          -> issn, numero, periodicidad

Socio              -> id, nombre, email, fechaAlta
Prestamo           -> id, idPublicacion, idSocio, fechaPrestamo, fechaDevolucion
```

Entregables

1. Diagrama de clases UML.
2. Script SQL de creación de tablas con claves primarias y foráneas.
3. Código Java organizado en paquetes: modelo, repositorio, servicio, excepciones, util.
4. Una clase Main que demuestre:
    - Registrar 3 libros y 2 revistas.
    - Registrar 2 socios.
    - Realizar un préstamo y consultarlo por ID.
    - Intentar prestar un libro ya prestado y capturar *LibroNoDisponibleException*.
    - Listar el catálogo ordenado por título.
    - Mostrar el contenido de la *Cache* tras varias consultas.